Sprint 3 – Week 2 Progress Report

SWENG 894 Capstone

MicroMatch

GitHub Repository Link

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace designed to connect students with short-term, skill-based professional opportunities. The platform enables:

- Organizations to post structured, well-defined micro-projects.
- Students to browse and discover opportunities.
- Students to apply for projects.
- Organizations to manage and review applications.
- A controlled workflow to manage project participation.
- End-to-end completion of projects through deliverables, review, and feedback.

The system architecture follows a layered full-stack model:

- **Frontend:** React-based SPA (Single Page Application)
- **Backend:** FastAPI REST API
- **Database:** PostgreSQL (relational persistence)
- **Authentication:** JWT-based stateless authentication
- **Authorization:** Role-Based Access Control (RBAC)

The platform supports two primary user roles:

- Student
- Organization

1.2 Sprint 3 Context

Sprint 3 focuses on completing the core system lifecycle, including:

- ➢ Application → Deliverable → Review → Completion → Feedback

Week 1 implemented:

- Deliverable submission (FR-10)
- Deliverable review (FR-11)

Week 2 extends functionality to:

- Project completion (FR-12)
- Feedback system (FR-13)

- Admin moderation (FR-14)

1.3 Functional Requirements to User Stories Mapping

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8
FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10
FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14
FR-15: The system shall log system actions.	US-15

FR-16: The system shall notify users of key events including application decisions, deliverable reviews, and project completion.	US-16
FR-17: The system shall allow users to send and receive messages within the context of a project.	US-17
FR-18: The system shall provide users with basic analytics including counts of applications and completed projects.	US-18
FR-19: The system shall allow profiles to display skills, completed projects, and ratings.	US-19

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint 3 – Week 2 Overview

2.1 Sprint Goal

The goal of Sprint 3 – Week 2 is:

- To complete the project lifecycle after deliverable review by implementing:
  - Project completion (FR-12 / US-12)
  - Feedback system (FR-13 / US-13)
  - Admin moderation (FR-14 / US-14)

These features finalize the operational lifecycle of MicroMatch and introduce governance and post-completion interaction.

3. Sprint Planning

3.1 Groomed Product Backlog

Priority Definitions

- High: Core MVP functionality required for end-to-end workflow.

- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	High	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	3	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure
US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern
US-16	Notifications	Medium	5	Sprint 4	User awareness
US-17	Messaging	Medium	8	Sprint 4	Communication
US-18	Analytics	Low	5	Sprint 4	Insights
US-19	Profile Enhancements	Low	3	Sprint 4	UX Improvements

3.2 Sprint Backlog – Week 2 Commitment

Based on my capacity, Week 2 included:

User Story	Description	Story Points
US-12	Mark Project Complete API	3
US-13	Feedback Submission API	5
US-13	Feedback UI Integration	3
US-14	Admin Moderation API	5

Total Commitment: 16 Story Points

Capacity Planning and Velocity Justification

As a single-developer team, historical velocity across previous sprints averaged:

- Sprint 1: ~16 story points
- Sprint 2: ~14 story points
- Sprint 3 Week 1: 16

This establishes a consistent velocity range of 15–18 story points per week.

Sprint 3 Week 2 Complexity Factors

Week 2 introduce:

- Workflow state transition across multiple entities (Applications → Deliverables → Projects)
- Role-based logic for **three roles** (Student, Organization, Admin)
- New database relationships for feedback
- Admin-level system access controls
- Frontend interfaces for completion, feedback, and moderation
- Integration with previously completed deliverable workflow

Commitment Justification

A commitment of 16 story points was selected because:

- It matches historical velocity
- Work is evenly distributed between backend, frontend, and integration
- The lifecycle features depend directly on Week 1 functionality, making integration efficient

- Scope remains controlled without introducing cross-cutting concerns (logging deferred to Week 3)

This ensures steady progress without risking quality or overcommitment.

3.3 Sprint 3 Backlog Structure

Sprint 3 was divided into three weekly development phases.

Week 1 – Deliverable Workflow

Primary Focus:

- Deliverable submission
- Deliverable review

Completed Issues

Issue	Description	Story Points	User Stories	Status
#64	Submit Deliverable API	5	US-10	Done
#65	Deliverable UI	3	US-10	Done
#66	Review Deliverable API	3	US-11	Done
#67	Deliverable Dashboard	3	US-11	Done
Supporting Work	Integration/Validation	2	US-10/US-11	Done

Week 1 Total Commitment: 16 Story Points

Week 2 – Completion & Platform Features

Primary Focus:

- Completing lifecycle
- Adding user-facing features

Completed Issues

Issue	Description	Story Points	Status
#68	Mark Project Complete API	3	Done
#69	Feedback Submission API	5	Done
#80	Feedback UI Integration	3	Done
#70	Admin Moderation API	5	Done

Week 2 Total Commitment: 16 Story Points

Week 3 — System Quality & Stability

Primary Focus:

- System-wide reliability
- Final validation

Planned Issues

Issue	Description	Story Points
#71	Logging Middleware	8
#81	End-to-End Workflow Testing	5
#82	Bug Fixes & UI Polish	3

Week 3 Total Commitment: 16 Story Points

4. UI Design

Wireframes were created for US-10 and US-15.

4.1 Submit Deliverables Dashboard (US-10)

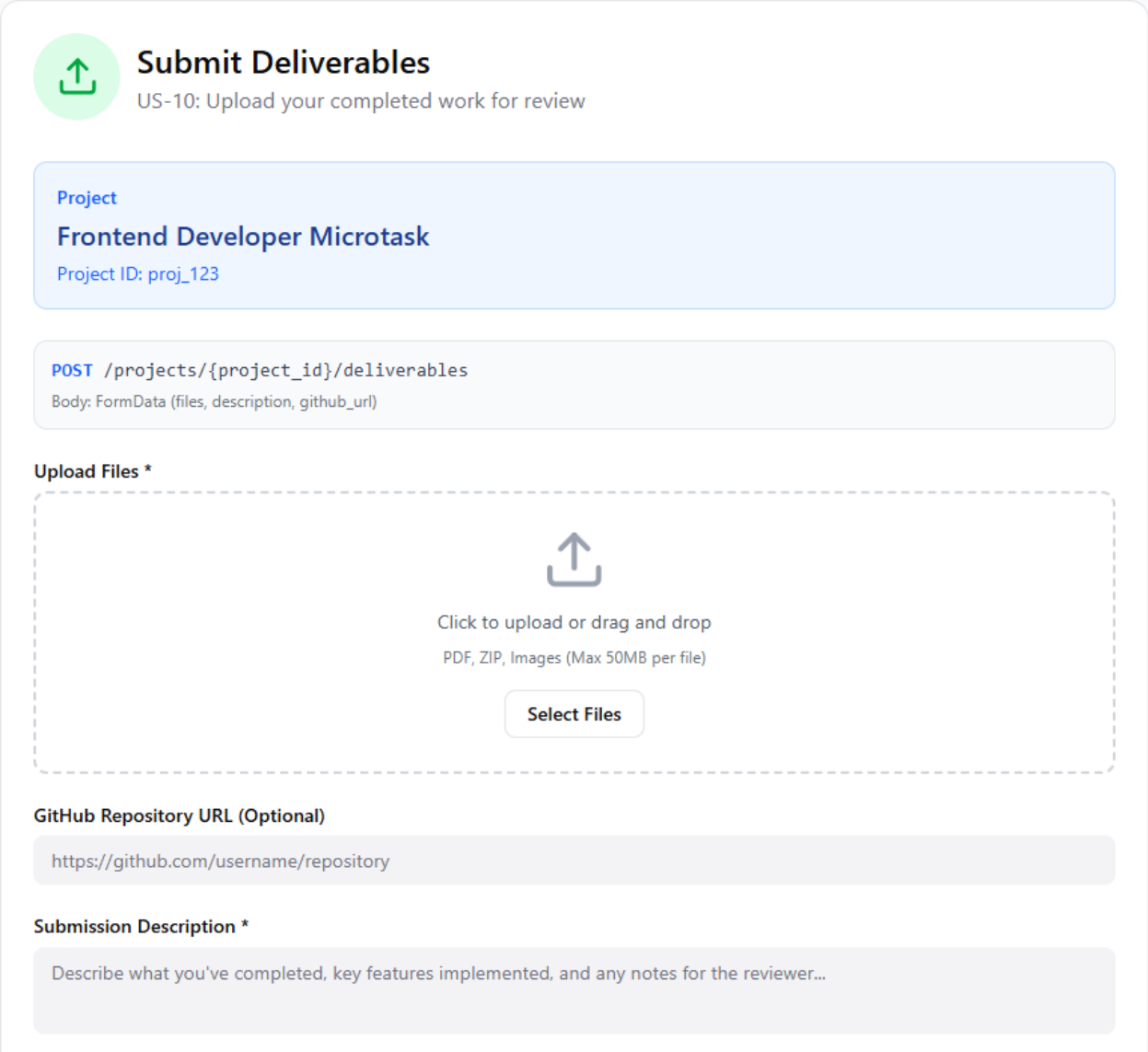


Figure 1 shows the wireframe for US-10 (Submit Deliverables).

Description and Design Rationale

The Submit Deliverables page enables authenticated student users to upload their completed project work for organization review. This interface provides students with a structured completion flow, allowing them to submit multiple file types, code repositories, and comprehensive descriptions of their work. The submission triggers the review process (US-11) and is a prerequisite for project completion (US-12).

Layout Structure

1. Project Information Header
    a. Project Title (read-only display)
    b. Project ID (read-only)
    c. Current project status
2. File Upload Section
    a. Drag-and-drop upload zone
    b. File type and size restrictions display
    c. Uploaded files list with preview
    d. File removal capability
3. Repository Link Section
    a. GitHub/GitLab URL input (optional)
    b. URL validation indicator
4. Description Section
    a. Rich text area for submission notes
    b. Character count indicator
    c. Required field validation
5. Submission Actions
    a. Primary "Submit Deliverables" button
    b. Secondary "Save Draft" option

Usability Considerations

Visual Upload Feedback:

- Drag-and-drop zone with hover state changes
- Real-time file upload progress indicators
- Clear file type acceptance messaging

File Management:

- Individual file preview cards showing name, size, and type
- Easy removal with X button for each file
- Total upload size calculation display

Submission Guidelines:

- Inline help text explaining submission requirements
- File size limits clearly communicated (50MB per file)
- Acceptable file formats listed prominently

Draft Functionality:

- Save draft capability prevents work loss
- Auto-save notification system
- Draft recovery on page revisit

Submission Confirmation:

- Clear success message post-submission
- Next steps information displayed
- Non-editable state after submission

Validation & Error Handling

Client-Side Validation:

- File size validation before upload (max 50MB per file)
- File type validation against whitelist
- URL format validation for repository links
- Required field checking (description + at least one file or URL)

Server-Side Error Handling:

- Upload timeout handling with retry option
- Duplicate submission prevention
- Virus scanning failure notifications
- Storage quota exceeded messaging

Empty State:

- Clear messaging if no files selected: "Please upload at least one file or provide a GitHub URL"

Error Messages:

- Displayed inline below affected fields
- Non-blocking warnings for optional fields
- Critical errors prevent submission with clear resolution steps

Security Alignment

- **Accessible only to:** Students assigned to the project
- JWT authentication required

- **Project assignment validation:** Students can only submit to projects where they have been accepted
- **File scanning**: All uploads scanned for malware before storage
- **File type restrictions:** Executable files blocked
- **Size limits enforced:** Prevent storage abuse
- **Rate limiting:** Prevent spam submissions

Alignment With Backend Design

This UI maps directly to:

Endpoint:

- POST /projects/{project_id}/deliverables

Response Schema Includes:

- deliverable.id
- deliverable.status (initially "submitted")
- deliverable.submission_timestamp
- deliverable.file_urls[]

State Transitions:

- Project status: "in-progress" → "deliverables-submitted"
- Deliverable status: null → "submitted"

4.2 Review Deliverables Dashboard (US-11)

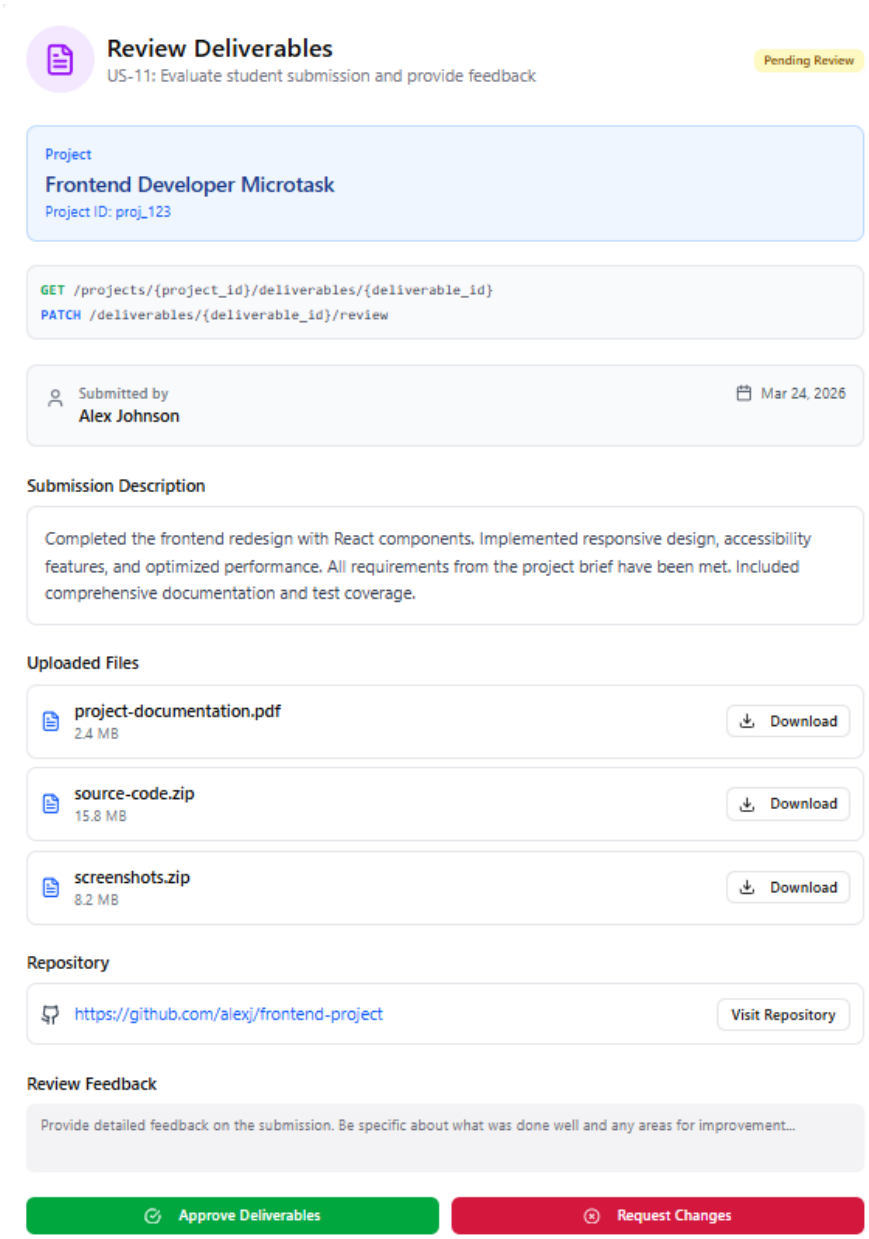


Figure 2 shows the wireframe for US-11 (Review Deliverables).

Description and Design Rationale

The Review Deliverables page enables authenticated organization users to evaluate student submissions and provide feedback.

This interface allows organizations to download and review submitted work, provide constructive feedback, and approve or request changes. The review decision is a

mandatory prerequisite for project completion (US-12) and ensures quality control before marking work complete.

Layout Structure

The interface is structured into six primary sections:

1. Header Section
    a. Project title and ID
    b. Current deliverable status badge
    c. Submission metadata (student name, date)
2. Student Information Card
    a. Student name
    b. Submission timestamp
    c. Student ID reference
3. Submission Content Display
    a. Description text (read-only, full display)
    b. Submitted files list with download buttons
    c. Repository link (if provided) with external link button
4. Feedback Section
    a. Multiline text area for review comments
    b. Character count indicator
    c. Required when rejecting
5. Review Actions
    a. "Approve Deliverables" (green CTA)
    b. "Request Changes" (destructive CTA)
    c. Both trigger confirmation modals
6. Validation Requirements Panel
    a. Checklist of review criteria
    b. Completion validation rules
    c. Next steps information

Usability Considerations

File Access:

- Individual download buttons for each file
- "Download All" bulk option for efficiency
- File preview capability for common formats (PDF, images)
- GitHub repository opens in new tab

Status Clarity:

- Large status badge at top of page
- Color-coded status indicators (Pending Review = Yellow, Approved = Green, Rejected = Red)
- Status change history display

Feedback Requirements:

- Feedback textarea expands on focus
- Character count helps set expectations
- Warning displayed if rejecting without feedback
- Feedback examples or templates available

Decision Confirmation:

- Modal confirmation for both approve and reject actions
- Summary of consequences displayed
- Ability to cancel before confirming
- No accidental decisions possible

Post-Decision State:

- Buttons disabled after decision
- Clear message about next steps
- Link to mark complete (US-12) if approved

Validation & Error Handling

Access Validation:

- Only project owner can review deliverables
- Unauthorized access returns 403 error with message

State Validation:

- Cannot review deliverables that have already been reviewed
- Invalid state transitions prevented (can't approve twice)

Feedback Requirements:

- Rejection requires feedback text
- Client-side validation before submission
- Minimum character count enforced (50 characters)

File Access Errors:

- File not found handling with retry option
- Download failure recovery
- Expired link messaging with refresh option

Empty State:

- If no deliverables submitted: "No deliverables have been submitted yet. Waiting for student submission."

Security Alignment

- **Accessible only to:** Organizations that own the project
- JWT authentication required
- **Ownership validation:** Organizations can only review projects they created
- **File access control:** Presigned URLs with expiration
- **Audit trail:** All review actions logged (US-15)
- **No modification:** Original files cannot be altered
- **Feedback visibility:** Sent to student, logged for compliance

Alignment With Backend Design

This UI maps directly to:

Endpoints:

- GET /projects/{project_id}/deliverables/{deliverable_id}
- PATCH /deliverables/{deliverable_id}/review

GET Response Schema Includes:

- deliverable.id
- deliverable.files[] (array of file metadata)
- deliverable.github_url
- deliverable.description
- deliverable.submitted_by (student info)
- deliverable.submitted_at
- deliverable.status

PATCH Request Body:

```
{
  "status": "approved" | "rejected",
  "feedback": "Detailed review feedback"
}
```

State Transitions:

- Deliverable status: "submitted" → "approved" OR "rejected"
- If approved: Project becomes eligible for completion (US-12)
- If rejected: Student notified to resubmit

4.3 Mark Project Complete (US-12)

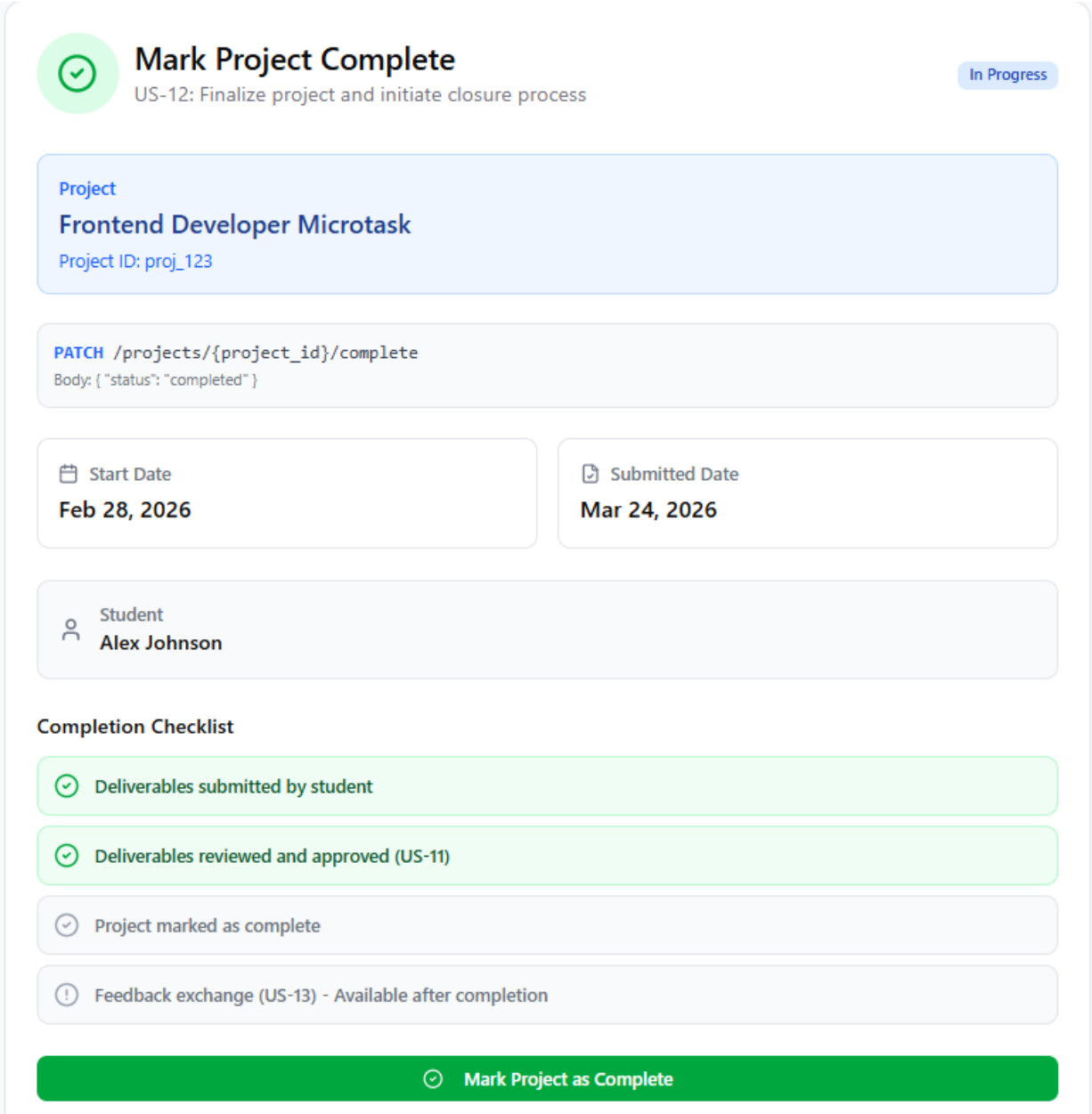


Figure 3 shows the wireframe for US-12 (Mark Project Complete).

Description and Design Rationale

The Mark Complete page enables authenticated organization users to finalize a project after deliverables have been approved. This interface provides the project closure mechanism, ensuring all prerequisites are met before marking a project permanently complete. This is a one-way state transition that triggers the feedback system (US-13) and updates project status across the platform.

Layout Structure

The interface is structured into five primary sections:

1. Header Section
    a. Project title and ID
    b. Current project status badge (In Progress → Completed)
    c. Completion timestamp display
2. Project Timeline Cards
    a. Start date display
    b. Submission date display
    c. Duration calculation
    d. Key milestone dates
3. Student Information Card
    a. Student name
    b. Student ID reference
    c. Contact information
4. Completion Checklist
    a. Deliverables submitted (✓)
    b. Deliverables approved (✓ or ✗)
    c. Project marked complete (pending)
    d. Feedback exchange available (future state)
    e. Visual checkmarks for completed items
5. Completion Actions
    a. Primary "Mark Project as Complete" button
    b. Disabled state if prerequisites not met
    c. Confirmation modal trigger

Usability Considerations

Prerequisites Visibility:

- Clear checklist showing completion requirements
- Visual indicators (green checkmarks) for met requirements
- Blocking message if deliverables not approved
- Prerequisite order clearly communicated

One-Way Decision Clarity:

- Warning that completion cannot be undone
- Modal confirmation required before finalizing
- Explanation of consequences displayed

Button States:

- Disabled state with tooltip explaining why
- Enabled only when all prerequisites met
- Clear CTA text: "Mark Project as Complete"

Post-Completion State:

- Success message with checkmark icon
- Next steps information (feedback opportunity)
- Link to feedback page (US-13)
- Project status permanently updated

Information Density:

- Timeline information helps verify completion timing
- Student information ensures correct project
- Checklist provides clear progress visualization

Validation & Error Handling

Prerequisites Validation:

- Deliverables must be in "approved" state
- Client-side check before enabling button
- Server-side validation prevents invalid completion
- Clear error message if prerequisites not met

Access Control Errors:

- Only project owner can mark complete
- 403 error with explanation if wrong user
- Redirect to appropriate page if unauthorized

State Conflict Handling:

- Cannot mark already-completed project
- Duplicate completion prevention
- Idempotent operation (safe to retry)

Network Error Handling:

- Retry mechanism for failed requests
- Loading state prevents double-submission
- Rollback capability if server error occurs

Empty State:

- If no deliverables: "Cannot mark complete without approved deliverables"

Security Alignment

- **Accessible only to:** Organizations that own the project
- JWT authentication required
- **Ownership validation:** Project ownership verified
- **State validation:** Prerequisites checked server-side
- **Irreversible action:** Logged and timestamped (US-15)
- **Notification trigger:** Student receives completion notification
- **Audit trail:** Completion event recorded with actor

Alignment With Backend Design

This UI maps directly to:

Endpoint:

- PATCH /projects/{project_id}/complete

Prerequisite Checks (Server-Side):

- Deliverable exists AND deliverable.status = "approved"
- Project.status = "in-progress"
- Requesting user owns project

Response Schema Includes:

- project.id
- project.status (now "completed")
- project.completed_at (timestamp)
- project.completed_by (organization user ID)

State Transitions:

- Project status: "in-progress" → "completed" (permanent)
- Triggers feedback availability (US-13)
- Updates student dashboard (auto-sync)

Side Effects:

- Student notification sent
- Feedback system unlocked
- Project removed from active queues
- Added to completion history

4.4 Feedback Dashboard (US-13)

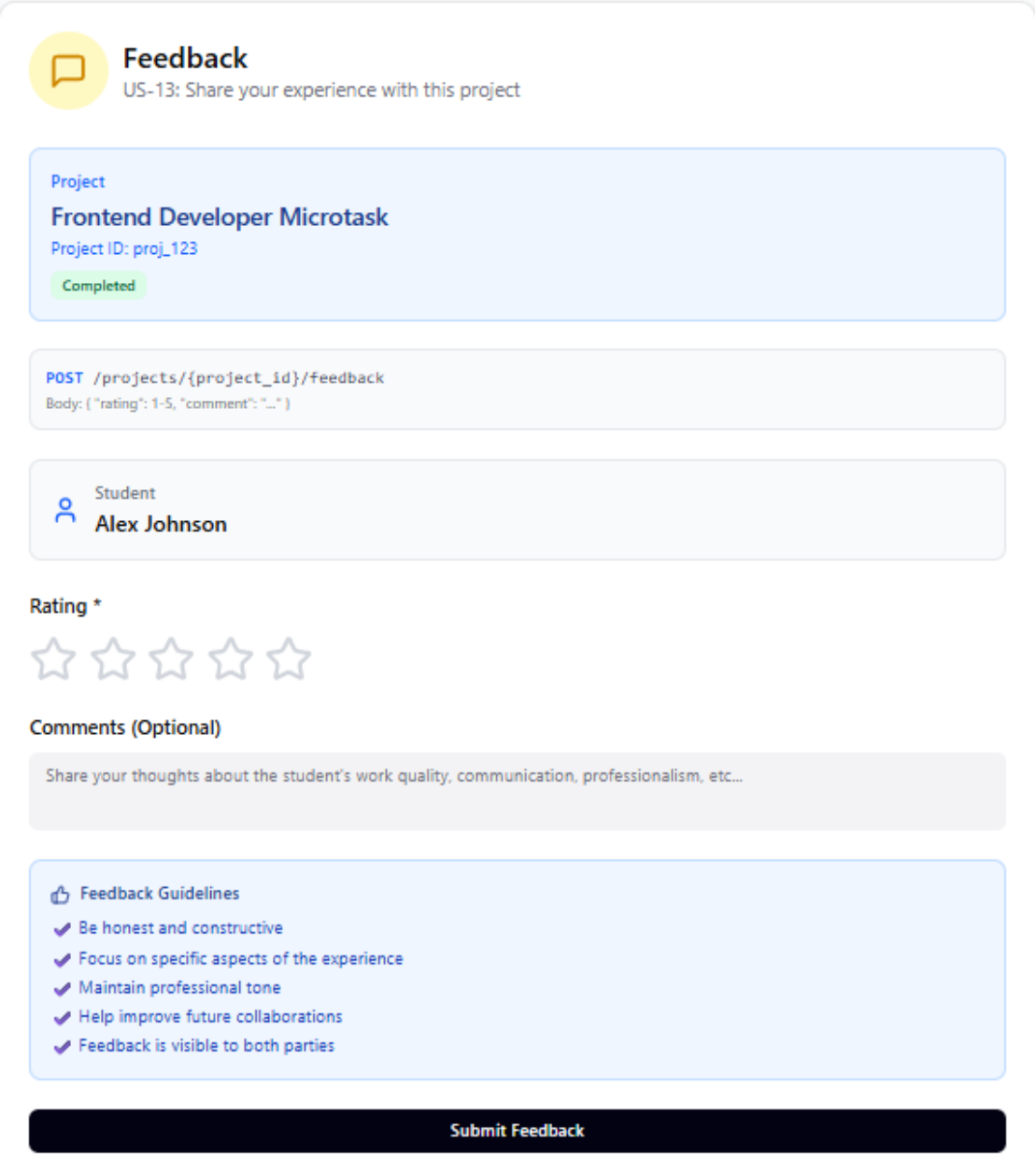


Figure 4 shows the wireframe for US-13 (Feedback Dashboard).

Description and Design Rationale

The Feedback page enables both students and organizations to provide ratings and comments about their project collaboration experience. This post-completion feature allows both parties to share their experience, contributing to user reputation scores and improving future matching. The feedback system is only available after project completion (US-12) and helps maintain platform quality through peer review.

Layout Structure

The interface is structured into six primary sections:

1. Header Section
    a. Project title and ID
    b. Project status badge (Completed)
    c. Feedback submission status
2. Partner Information Card
    a. Partner name (Organization or Student)
    b. Partner role indicator with icon
    c. Project relationship context
3. Rating Section
    a. Interactive 5-star rating system
    b. Hover state preview
    c. Rating label (Poor, Fair, Good, Very Good, Excellent)
    d. Required field indicator
4. Comments Section
    a. Multiline text area (optional)
    b. Placeholder text tailored to user role
    c. Character counter
    d. Constructive feedback guidelines
5. Guidelines Panel
    a. Feedback best practices
    b. Tone and professionalism reminders
    c. Visibility notice (feedback shown to both parties)
6. Submission Actions
    a. "Submit Feedback" button
    b. Success confirmation screen post-submission
    c. One-time submission enforcement

Usability Considerations

Rating Interaction:

- Large, touch-friendly star buttons
- Hover state shows rating preview
- Clear rating labels for each value
- Visual feedback on selection (filled stars)

Contextual Placeholders:

- Student placeholder focuses on organization experience
- Organization placeholder focuses on student performance
- Examples help guide appropriate feedback

Feedback Guidelines:

- Inline tips promote constructive comments
- Professional tone encouraged
- Specific examples over general statements
- Visible to both parties notice

Single Submission:

- Cannot edit or resubmit after submission
- Clear confirmation of submitted feedback
- Summary display of what was submitted
- Return to dashboard option

Post-Submission State:

- Thank you message with checkmark icon
- Display of submitted rating and comment
- Explanation of how feedback contributes to reputation
- Navigation back to dashboard

Validation & Error Handling

Prerequisites Validation:

- Only available after project completion (US-12)
- Access denied if project status ≠ "completed"
- Clear message: "Feedback available after project completion"

Rating Requirement:

- Rating must be selected (1-5 stars)
- Client-side validation prevents submission without rating
- Error message: "Please provide a rating before submitting"

Duplicate Prevention:

- Cannot submit feedback twice for same project
- Check performed on page load
- If already submitted, show summary instead of form

Character Limits:

- Comment length: 0-2000 characters
- Real-time character counter
- Graceful truncation if exceeded

Network Error Handling:

- Retry mechanism for failed submissions
- Loading state prevents double-submission
- Draft saved locally if connection lost

Security Alignment

- **Accessible to:** Both students and organizations involved in completed project
- JWT authentication required
- **Project participation validation:** Only project participants can submit feedback
- **One submission per user per project:** Enforced server-side
- **Visibility:** Feedback visible to both parties (student and organization)
- **Moderation:** Flagged feedback reviewed by admins (US-14)
- **Reputation impact:** Aggregate ratings affect user profiles

Alignment With Backend Design

This UI maps directly to:

Endpoint:

- ➢ POST /projects/{project_id}/feedback

Prerequisites (Server-Side):

- Project.status = "completed"
- User is project participant (student OR organization)
- No existing feedback from this user for this project

Response Schema Includes:

- feedback.id
- feedback.rating
- feedback.comment
- feedback.created_at

- feedback.created_by (user ID)
- feedback.project_id

State Transitions:

- Feedback status: null → "submitted"
- User's feedback_given flag set to true for this project

Side Effects:

- Recipient receives notification of new feedback
- Aggregate rating updated on recipient profile
- Feedback visible to both parties immediately
- Contributes to user reputation score

4.5 Admin Moderation Dashboard (US-14)

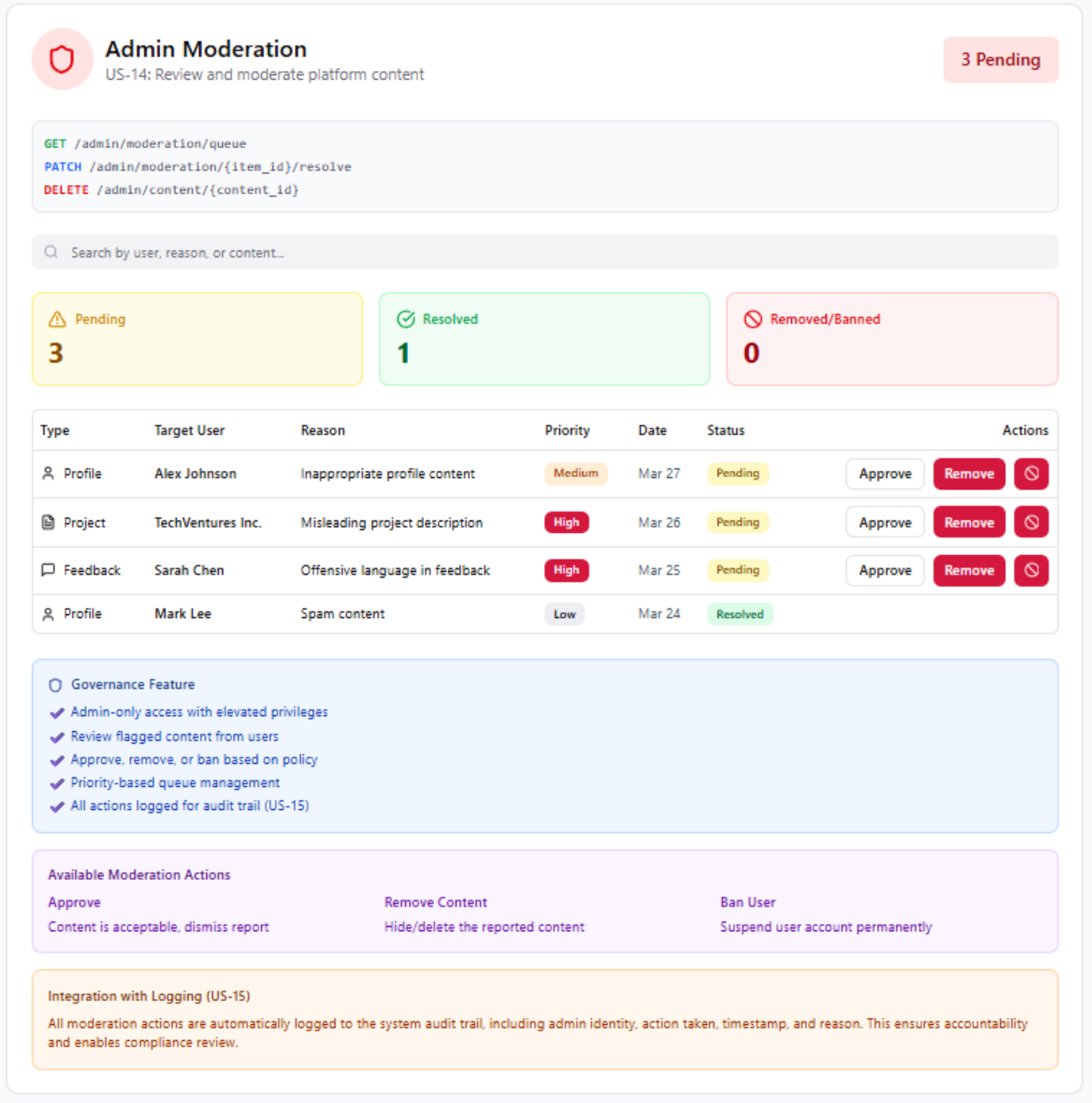


Figure 5 shows the wireframe for US-14 (Admin Moderation Dashboard).

Description and Design Rationale

The Admin Moderation page provides platform administrators with tools to review and moderate flagged content. This governance feature enables admins to maintain platform quality by reviewing user-reported content, enforcing community guidelines, and taking action on policy violations. All moderation actions are automatically logged (US-15) for accountability and compliance.

Layout Structure

The interface is structured into six primary sections:

1. Header Section
    a. Admin-only badge/indicator
    b. Pending items count (prominent badge)
    c. Page title and description
2. Search and Filters Bar
    a. Text search across all fields
    b. Level filter dropdown
    c. Action type filter dropdown
    d. Date range selector (future enhancement)
3. Statistics Dashboard
    a. Pending count (yellow)
    b. Resolved count (green)
    c. Removed/Banned count (red)
    d. Visual separation with color coding
4. Moderation Queue Table
    a. Content type icon and label
    b. Target username
    c. Reported reason
    d. Priority badge (High, Medium, Low)
    e. Report date
    f. Status badge
    g. Actions column (Approve, Remove, Ban buttons)
5. Governance Information Panel
    a. Access control information
    b. Review process explanation
    c. Available actions summary
    d. Integration with logging (US-15)
6. Confirmation Models
    a. Action-specific confirmation dialogs
    b. Consequence explanation
    c. Logging notice
    d. Cancel and Confirm options

Usability Considerations

Priority Visualization:

- High priority items styled in red
- Medium priority in orange
- Low priority in gray
- Sortable by priority for triage

Content Type Icons:

- User reports (person icon)
- Project reports (document icon)
- Feedback reports (message icon)
- Quick visual scanning enabled

Action Clarity:

- Three distinct actions available: Approve, Remove, Ban
- Color-coded action buttons (green, yellow, red)
- Clear labels explaining each action
- Disabled state after action taken

Filtering and Search:

- Real-time search across multiple fields
- Combined filters for complex queries
- Filter persistence across page refresh
- Clear filter reset option

Batch Operations:

- Select multiple items for bulk action (future enhancement)
- Priority-based sorting for efficiency
- Keyboard shortcuts for power users

Validation & Error Handling

Access Control:

- Admin role required (enforced server-side)
- 403 Forbidden if non-admin attempts access
- Clear "Admin Only" messaging

Action Validation:

- Cannot act on already-resolved items
- Server-side validation prevents duplicate actions

- Invalid state transitions blocked

Confirmation Requirements:

- All actions require explicit confirmation
- Modal prevents accidental decisions
- "Are you sure?" verification

Error Handling:

- Network errors show retry option
- Stale data refresh mechanism
- Concurrent modification detection

Empty State:

- If queue empty: "No pending moderation items. All clear!"
- Celebratory message for empty queue

Security Alignment

- **Accessible only to:** Users with role = "Admin"
- JWT authentication required with admin scope
- **Elevated privileges:** Full content access and modification rights
- **Audit trail:** All actions logged with admin identity (US-15)
- **Accountability:** Cannot delete log entries
- **Two-factor authentication:** Required for admin actions (future)
- **Rate limiting:** Prevents automated abuse
- **IP logging:** Admin actions tracked by IP address

Alignment With Backend Design

This UI maps directly to:

Endpoints:

- GET /admin/moderation/queue
- PATCH /admin/moderation/{item_id}/resolve
- DELETE /admin/content/{content_id}
- POST /admin/users/{user_id}/ban

GET Queue Response Schema Includes:

{

```
  "items": [
    {
      "id": "mod_001",
      "type": "user" | "project" | "feedback",
      "target_id": "user_123",
      "target_name": "User Name",
      "reported_by": "user_456",
      "reason": "Violation description",
      "priority": "high" | "medium" | "low",
      "status": "pending" | "resolved" | "removed",
      "created_at": "timestamp"
    }
  ],
  "pagination": {...}
}
```

Action Request Bodies:

Approve:

```
{
  "action": "approve",
  "resolution_note": "Content is acceptable"
}
```

Remove Content:

```
{
  "action": "remove",
  "reason": "Policy violation"
}
```

Ban User:

```
{
  "action": "ban_user",
  "reason": "Repeated violations",
  "permanent": true
}
```

State Transitions:

- Moderation item: "pending" → "resolved" | "removed" | "user-banned"
- Content visibility: visible → hidden (if removed)
- User status: active → banned (if banned)

Side Effects:

- User notification sent for ban or content removal
- Moderation action logged with admin ID, timestamp, reason (US-15)
- Banned users cannot login
- Removed content hidden from platform
- Aggregate statistics updated

4.6 Logging Dashboard (US-15)

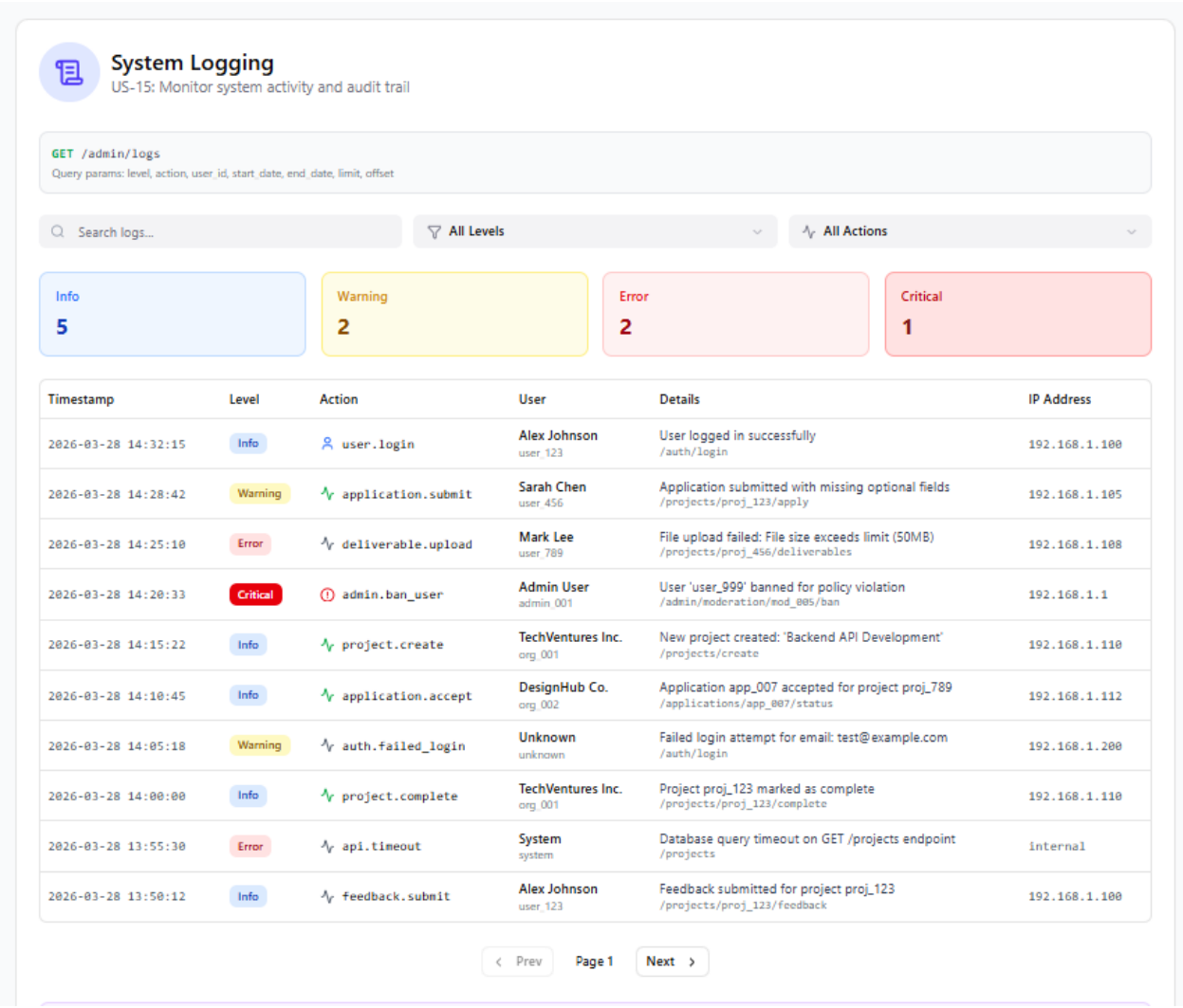


Figure 6 shows the wireframe for US-15 (Logging Dashboard).

Description and Design Rationale

The Logging page provides platform administrators with comprehensive visibility into all system activities through a searchable, filterable audit trail. This cross-cutting concern captures all user actions, API calls, system events, authentication attempts, and administrative actions. The logging system supports debugging, security auditing, compliance requirements, and provides the accountability layer for all platform operations, including admin moderation actions (US-14).

Layout Structure

The interface is structured into six primary sections:

1. Header Section
    a. System-wide activity indicator
    b. Log count summary
    c. Time range selector
2. Filters and Search Bar
    a. Text search across all log fields
    b. Log level filter (Info, Warning, Error, Critical)
    c. Action type filter (User, Admin, Project, Application, Auth)
    d. Date range picker (future enhancement)
3. Statistics Dashboard
    a. Info count (blue)
    b. Warning count (yellow)
    c. Error count (red)
    d. Critical count (dark red)
    e. Real-time updates
4. Logs Table
    a. Timestamp (sortable)
    b. Log level badge
    c. Action type with icon
    d. User name and ID
    e. Details/message
    f. IP address
    g. Endpoint reference
5. Pagination Controls
    a. Previous/Next buttons
    b. Page number display
    c. Items per page selector
    d. Total count indicator
6. Export and Analysis Tools
    a. Export to CSV option
    b. Advanced filters toggle
    c. Time-based grouping views
    d. Anomaly detection alerts (future)

Usability Considerations

Real-Time Updates:

- Auto-refresh option for live monitoring

- New log indicator when refresh available
- Pause auto-refresh during detailed inspection

Color-Coded Severity:

- **Info:** Blue (informational)
- **Warning:** Yellow (attention needed)
- **Error:** Red (failure occurred)
- **Critical:** Dark red (system-level issue)
- Consistent with industry standards

Action Type Icons:

- **User actions:** Person icon
- **Admin actions:** Shield icon
- **Project/Application:** Activity icon
- **Authentication:** Lock icon
- Quick visual categorization

Filtering Efficiency:

- Multi-select filters for complex queries
- Filter combinations preserved
- Quick filter presets (e.g., "All Errors Today")
- Clear active filters display

Detailed Log View:

- Expandable rows for full details
- JSON payload viewer for API logs
- Stack trace display for errors
- Copy to clipboard functionality

Export Capabilities:

- CSV export for spreadsheet analysis
- Date range selection for exports
- Filter-aware exports (only visible logs)
- Batch export for large datasets

Validation & Error Handling

Access Control:

- Admin-only access enforced
- Read-only view (logs cannot be modified)
- 403 error if non-admin attempts access

Query Validation:

- Date range limits to prevent performance issues
- Maximum result set size (10,000 records)
- Invalid query parameter handling

Empty State:

- If no logs match filters: "No logs found matching your criteria"
- Suggestion to adjust filters or date range

Performance Protection:

- Pagination required for large datasets
- Query timeout protection (30 seconds)
- Rate limiting on log queries

Data Integrity:

- Logs cannot be deleted or modified
- Immutable audit trail
- Tampering detection (checksums)

Security Alignment

- **Accessible only to:** Users with role = "Admin"
- JWT authentication required with admin scope
- **Read-only access:** Logs immutable, preventing tampering
- **Comprehensive capture:** All actions logged, no gaps
- **PII handling:** IP addresses and user IDs logged but encrypted at rest
- **Retention policy:** Logs retained for 1 year minimum (compliance)
- **Audit the auditors:** Admin log access also logged
- **Export audit trail:** Log exports tracked

Alignment With Backend Design

This UI maps directly to:

Endpoint:

- GET /admin/logs

Query Parameters:

?level=info|warning|error|critical

&action=user|admin|project|application|auth

&user_id=user_123

&start_date=2026-03-01

&end_date=2026-03-31

&limit=20

&offset=0

&search=keyword

Response Schema Includes:

```
{
  "logs": [
    {
      "id": "log_001",
      "timestamp": "2026-03-28T14:32:15Z",
      "level": "info" | "warning" | "error" | "critical",
      "action": "user.login",
      "user_id": "user_123",
      "user_name": "Alex Johnson",
      "ip_address": "192.168.1.100",
      "endpoint": "/auth/login",
      "method": "POST",
      "status_code": 200,
      "details": "Descriptive message",
      "request_id": "req_xyz789",
```

```
    "duration_ms": 125
  }
 ],
 "pagination": {
  "total": 1523,
  "limit": 20,
  "offset": 0,
  "has_next": true
 },
 "statistics": {
  "info": 1200,
  "warning": 250,
  "error": 68,
  "critical": 5
 }
}
```


Logged Events Include:

1. **Authentication Events (US-6, US-7):**
    a. Login attempts (success/failure)
    b. Logout events
    c. Token refresh
    d. Password reset requests
2. **User Actions:**
    a. Profile updates
    b. Application submissions
    c. Deliverable uploads (US-10)
    d. Feedback submissions (US-13)
3. **Organization Actions:**
    a. Project creation
    b. Application reviews (US-8)

    c. Deliverable reviews (US-11)
    d. Project completion (US-12)
4. **Admin Actions (US-14):**
    a. Content moderation decisions
    b. User bans
    c. Content removal
    d. Policy enforcement
5. **System Events:**
    a. API errors and timeouts
    b. Database query performance issues
    c. External service failures
    d. Scheduled job executions

Log Levels Defined:

- **Info:** Normal operations, successful actions
- **Warning:** Potential issues, deprecated API usage, validation warnings
- **Error:** Failed operations, caught exceptions, user errors
- **Critical:** System failures, security incidents, data corruption

Cross-Cutting Implementation:

- Middleware captures all HTTP requests/responses
- Database triggers log data modifications
- Application code logs business logic events
- Security module logs authentication/authorization
- Background jobs log scheduled task execution

This logging system provides complete visibility across all Sprint 3 user stories (US-10 through US-15) and all previous sprints, ensuring accountability, debuggability, and compliance.

5. Definition of Done (Sprint 3)

The Definition of Done was refined for Sprint 3 and applied consistently to all backlog items.

A User Story is considered complete when:

Design

- Data model is designed and reviewed
- Database schema aligns with requirements
- API contract is clearly defined
- Endpoint request/response structures documented

Implementation

- Database migration successfully applied
- Backend endpoint implemented and functional
- Business logic correctly enforces requirements
- Role-Based Access Control (RBAC) enforced
- Input validation implemented (edge cases handled)
- Frontend components implemented (if applicable)
- Frontend successfully integrated with backend APIs

Testing

- Unit tests written for core logic
- Edge case scenarios covered
- API endpoints tested (success + failure cases)
- Manual functional testing completed
- No failing test cases

Quality & Integration

- Code coverage ≥ 90% for core business logic
- Code reviewed and approved
- Pull request merged into main branch
- No regression introduced to previous features
- System builds and runs without errors

Documentation

- API endpoints documented
- Code comments added where necessary
- User flow documented (if applicable)

Deployment Readiness

- Feature is deployable without breaking system
- Environment variables/configurations updated

6. Acceptance Criteria

6.1 US-12 – Mark Project Complete

Acceptance Criteria:

- Only authenticated organization users may mark a project complete
- Deliverable for the project must exist
- Deliverable must have status **accepted**
- Project must not already be completed
- Project status updates to **completed**
- Completion timestamp is recorded

Returns:

- 200 OK on success
- 400 Bad Request if deliverable not accepted or already completed
- 403 Forbidden if wrong role
- 404 Not Found if project not found

6.2 US-13 – Feedback Submission

Acceptance Criteria:

- Only authenticated users involved in the project may submit feedback
- Project must have status completed
- Feedback must include rating and text
- Duplicate feedback submissions prevented (one per user per project)
- Feedback linked to project and user

Returns:

- 201 Created on success
- 400 Bad Request if project not completed or duplicate feedback

- 403 Forbidden if unauthorized user
- 404 Not Found if project not found

6.3 US-14 – Admin Moderation

Acceptance Criteria:

- Only authenticated admin users may access moderation endpoints
- Admin can view all projects
- Admin can disable/remove inappropriate projects
- Changes reflect immediately in system
- Moderation actions logged (to be fully implemented in Week 3)

Returns:

- 200 OK on success
- 403 Forbidden if non-admin
- 404 Not Found if project does not exist

7. Source Code Development Progress

7.1 Repository

GitHub Repository: https://github.com/chazdj/MicroMatch

Project board integrated with GitHub Issues for traceability between user stories and commits.

7.2 Summary of Contributions

Backend Contributions

- Implemented PUT /projects/{id}/complete
- Implemented POST /projects/{id}/feedback
- Implemented GET /admin/projects
- Implemented moderation controls
- Enforced state rules: accepted → completed → feedback

Frontend Contributions

- Completion button UI
- Feedback form UI
- Admin dashboard panel
- Role-based rendering for admin/org/student

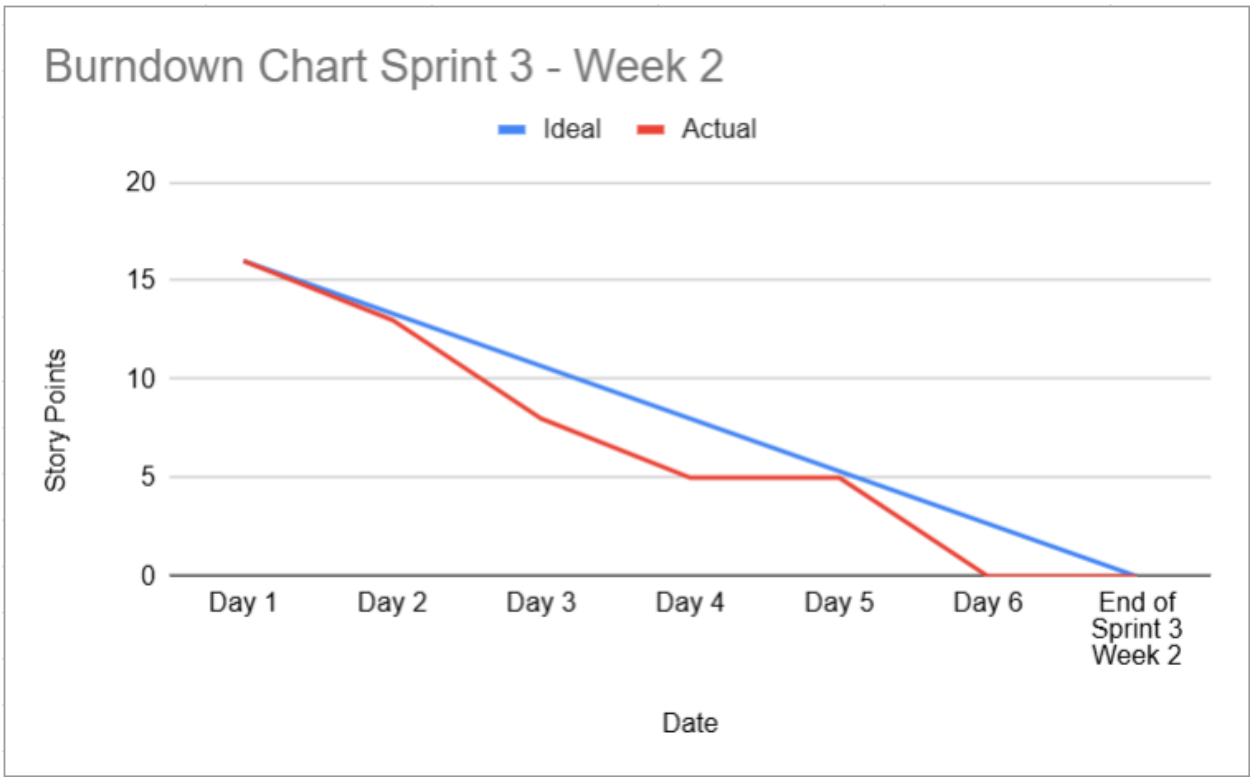
Integration Work

- Verified lifecycle:
  - Submitted → Reviewed → Completed → Feedback
- Manual end-to-end validation

7.3 Important Commits

Commit	Related Issue	User Story
feat: project completion endpoint	#68	US-12
test: project completion endpoint	#68	US-12
feat: feedback endpoint	#69	US-13
test: feedback endpoint	#69	US-13
ui: feedback form	#80	US-13
feat: admin moderation	#70	US-14
test: admin moderation	#70	US-14

7.4 Burndown Chart



Sprint Capacity: 16 Story Points
Sprint Duration: 6 Development Days

Burndown Chart Analysis

- Work decreased linearly as features were implemented in logical order
- Backend-first approach reduced integration risk
- No scope changes occurred during the sprint
- Velocity aligned perfectly with Week 1 performance
- The stable burndown trend indicates accurate estimation and disciplined scope control

Velocity Alignment:

Sprint 1 average velocity: ~16 points
Sprint 2 average velocity: ~14 points
Sprint 3 Week 2 completed: 16 points

8. Software Testing

8.1 Requirement-to-Test Mapping

Functional Requirement	User Story	Test Case ID
FR-12	US-12	TC-US12-01 - TC-US12-03
FR-13	US-13	TC-US13-01 - TC-US13-03
FR-14	US-14	TC-US14-01 - TC-US14-03

8.2 User Acceptance Test Specification

All test cases are mapped to FR-12 through FR-14 ensuring full coverage of sprint scope.

TC-US12-01 – Successful Completion

Related Requirement: FR-12
User Story: US-12 – Mark Project Complete
Actor: Organization

Preconditions:

- Organization account exists in system
- Organization user is logged in
- A project exists owned by the organization
- A deliverable exists for that project
- Deliverable status is Accepted
- Project status is In Progress

Steps:

1. Open the MicroMatch application in a web browser.
2. Navigate to the login page.
3. Enter valid organization credentials and click Login.
4. From the dashboard, locate the list of active projects.
5. Click on the project with an accepted deliverable.
6. Verify the project page displays deliverable status as Accepted.
7. Locate the Mark Project Complete button.
8. Click the button.
9. Confirm the action in the confirmation dialog.
10. Observe the system sending a completion request to the backend.

Expected Result:

- HTTP 200 OK returned from server
- Project status changes to Completed
- Completion timestamp saved
- UI updates to show completed badge
- Completion button no longer visible or disabled

Postconditions:

- Project is permanently marked as completed in database
- Feedback functionality becomes available

TC-US12-02 – Prevent Completion if Not Reviewed

Related Requirement: FR-12
User Story: US-12 – Mark Project Complete
Actor: Organization

Preconditions:

- Organization user is logged in
- Project exists
- Deliverable exists with status Pending or Rejected

Steps:

1. Login as organization user.
2. Navigate to the project page.
3. Verify deliverable status is not Accepted.
4. Attempt to click Mark Project Complete.
5. Observe system response.

Expected Result:

- HTTP 400 Bad Request returned
- Error message indicating deliverable must be accepted
- Project status remains unchanged

Postconditions:

- Project remains In Progress

TC-US12-03 – Unauthorized Role

Related Requirement: FR-12
User Story: US-12 – Mark Project Complete
Actor: Student

Preconditions:

- Student account exists and is logged in
- Project exists with accepted deliverable

Steps:

1. Login as student.
2. Navigate to the project page.
3. Attempt to locate Mark Project Complete button.
4. Attempt API call manually if button not visible.

Expected Result:

- Button is not visible in UI
- HTTP 403 Forbidden if endpoint accessed directly

Postconditions:

- No change to project status

TC-US13-01 – Successful Feedback Submission

Related Requirement: FR-13
User Story: US-13 – Feedback System
Actor: Student

Preconditions:

- Student participated in the project
- Project status is Completed
- Student is logged in

Steps:

1. Open MicroMatch in browser.
2. Login as student.
3. Navigate to Completed Projects.
4. Select the completed project.
5. Locate the Feedback section.
6. Enter rating value.
7. Enter written feedback text.
8. Click Submit Feedback.
9. Observe request sent to backend.

Expected Result:

- HTTP 201 Created returned
- Feedback saved successfully
- Confirmation message displayed
- Feedback visible on project page

Postconditions:

- Feedback stored in database linked to user and project

TC-US13-02 – Prevent Feedback Before Completion

Related Requirement: FR-13
User Story: US-13 – Feedback System
Actor: Student

Preconditions:

- Project status is In Progress
- Student is logged in

Steps:

1. Login as student.
2. Navigate to project page.
3. Attempt to access feedback form.
4. Attempt to submit feedback.

Expected Result:

- Feedback form not accessible OR

- HTTP 400 Bad Request returned
- Error message shown

Postconditions:

- No feedback stored

TC-US13-03 – Duplicate Feedback Prevention

Related Requirement: FR-13
User Story: US-13 – Feedback System
Actor: Organization

Preconditions:

- Project is completed
- Feedback already submitted by this user

Steps:

1. Login as organization user.
2. Navigate to completed project.
3. Attempt to submit feedback again.

Expected Result:

- HTTP 400 Bad Request returned
- Message indicating feedback already submitted

Postconditions:

- Only original feedback remains stored

TC-US14-01 – Admin Views Projects

Related Requirement: FR-14
User Story: US-14 – Admin Moderation
Actor: Admin

Preconditions:

- Admin account exists
- Admin is logged in

Steps:

1. Open MicroMatch.
2. Login as admin.
3. Navigate to Admin Dashboard.
4. Click View All Projects.
5. Observe list of all projects in system.

Expected Result:

- HTTP 200 OK returned
- All projects displayed regardless of status

Postconditions:

- Admin able to select any project for moderation

TC-US14-02 – Admin Removes Project

Related Requirement: FR-14
User Story: US-14 – Admin Moderation
Actor: Admin

Preconditions:

- Admin logged in
- Project exists flagged for removal

Steps:

1. Login as admin.
2. Navigate to project list.
3. Select the inappropriate project.
4. Click Remove/Disable Project.
5. Confirm action.

Expected Result:

- HTTP 200 OK returned
- Project status changed to Disabled/Removed
- Project no longer visible to students or organizations

Postconditions:

- Project stored as disabled in database

TC-US14-03 – Unauthorized Access

Related Requirement: FR-14
User Story: US-14 – Admin Moderation
Actor: Student

Preconditions:

- Student account exists and logged in

Steps:

1. Login as student.
2. Attempt to navigate to admin dashboard URL directly.
3. Attempt to access admin API endpoint.

Expected Result:

- HTTP 403 Forbidden returned
- Access denied message shown

Postconditions:

- No access granted to admin features

9. Backlog Grooming

No changes were made to the sprint backlog or product backlog during Sprint 3 Week 2.

Sprint execution followed the planned structure defined in Week 1.

10. Risk Management

Risk	Mitigation Strategy
Completing project before review	Enforced state validation (must be accepted)
Feedback submitted prematurely	Project status check before feedback allowed
Duplicate feedback entries	Database constraint + validation logic
Unauthorized admin access	Strict RBAC enforcement
Integration conflicts with Week 1 features	Early integration testing
UI allowing invalid actions	Conditional rendering and disabled buttons

11. Sprint 3 Week 2 Outcome

Sprint 3 Week 2 successfully delivered:

- Project lifecycle completion
- Feedback system
- Admin governance panel

MicroMatch now supports the full project lifecycle from application → deliverable → review → completion → feedback, with administrative oversight.

The system is now functionally complete. Week 3 will focus on logging, stability, and final validation.